Ben Grubbs
GEOG-581

# Classification of Urban Canopy Cover with LiDAR and Multispectral Data

## Background

The presence of trees can enhance human, environmental, and ecological health in urban environments. Having access to accurate and up-to-date canopy cover maps is important for supporting sustainable urban planning efforts. LiDAR point clouds have been used for automated classification of buildings and trees, but this approach is somewhat limited in terms of distinguishing between trees and non-tree land cover types. Satellite imagery, such as the data obtained from Sentinel-2, can provide multispectral information that can be used to analyze vegetation and other land cover types (Ebadat et al., 2016). A machine learning classifier known as “Random Forest'' has gained popularity as a technique used for automated classification of land cover (Belgiu & Dragut, 2016). Random Forest is an ensemble classifier, where an ensemble classifier is defined as a “learning algorithm that constructs a set of classifiers and then classifies new data points by taking a weighted vote of their predictions” (Dietterich, 2000). Combining LiDAR data with Multispectral data can give us a large selection of variables that can be used for classification with a Random Forest approach (Ebadat et al., 2016).

## Project Objectives

The goal of this project was to extract data from LiDAR and Sentinel-2 multispectral imagery, and to use these data as “predictor variables” in a random forest classifier to classify tree and non-tree land cover. The project area was the Irvington Historic District neighborhood in Indianapolis,

Indiana. See **Figure 1** below for an overview of the project methods. **Figures 2 and 3** below show the Irvington Historic District project area.

## Methods

To train the classifier, reference data were collected using Collect Earth Online (CEO). The reference data consisted of 245 total plots, each 10 m in width. Each plot had multiple samples that consisted of points and polygons, which resulted in 542 total samples. Each sample contained a binary classification of “tree” and “non-tree” land cover type. This data was then exported as a CSV file and uploaded as an asset in Google Earth Engine (GEE).

LiDAR variables included slope, aspect, Canopy Height Model (CHM), and Digital Elevation Model (DEM). LiDAR point cloud data were retrieved from USGS to extract slope and aspect. The CHM and DEM were obtained from the Indiana Geographic Information Council. LiDAR data was processed using CloudCompare and QGIS, and each variable was then uploaded as an asset in GEE.

Sentinel-2 variables included the Normalized Difference Vegetation Index (NDVI), Enhanced Vegetation Index (EVI), Leaf Area Index (LAI), Normalized Difference Water Index (NDWI), Normalized Difference Moisture Index (NDMI), Normalized Difference Built-up Index (NDBI), and an RGB composite consisting of bands 2, 3, and 4. These variables were extracted in Google Earth Engine and saved as assets.

All LiDAR and Sentinel variables were merged into a multiband image in GEE, where each individual band contained one of the variables (i.e. NDVI = B1, EVI = B2, etc.). **Figures 4, 5 and 6** below show the code used for analysis

in GEE. **Figure 7** shows a visual example of some of the LiDAR and Sentinel variables that were used as predictor variables.

## Results

The classifier was trained using the reference data and was then applied to the predictor variables in the multiband image to classify the land cover as “tree” and “non-tree”. An accuracy assessment was performed to evaluate the training data, validation data, generate a confusion matrix, and calculate user accuracy, and producer accuracy. The overall training accuracy was 0.96, overall validation accuracy was 0.88, and user accuracy was 0.88. For producer accuracy, the non-tree class had an accuracy of 0.72, and the tree class had an accuracy of 0.97. See **Figure 8** below for the results of the classified map and see **Figure 9** for Google satellite imagery for comparison. See **Figures 10 and 11** for a zoomed in comparison of the classified map and the satellite imagery. For the graphic containing the confusion matrix, validation error matrix, and resubstitution error matrix results, see **Figures 12-19**.

## Discussion

The classifier appeared to do well at identifying tree cover with a 97% accuracy, but not as well at identifying non-tree cover with a 72% accuracy. To summarize, 97% of the time, the classifier predicted the presence of a tree in a location where the validation data agreed with the classification, but 3% of the time the validation data disagreed and indicated that the land cover should be “non-tree” instead of “tree”. Additionally, 72% of the time the classifier predicted the presence of “non-tree” land cover at a location where the validation data agreed, but 28% of the time the validation data

disagreed and indicated that the land cover should be “tree” instead of “non-tree”. The overall accuracy of the classified image is around 88.6%. This is a reasonably accurate starting point which could be improved upon further with future research.

To improve upon this project, I would recollect the training data, ensuring that the most statistically sound sample design is being used. I would also collect more training data, ideally having 1,000 or more reference points. Additionally, I would compare the results from using different combinations of variables. It is possible that some of the current variables are redundant or are not informing the classifier as expected, though I do feel that each of the variables chosen are important and informative.

Through this project (and some of the troubleshooting and project exploration), I have gained programming experience using JavaScript, Python, and excel. I have learned about LiDAR data sources and how to locate, download, process, and analyze LiDAR data. I have also learned about the Sentinel-2 program and the large selection of spectral bands and high spatial resolution it can offer. Additionally, I learned about the statistical/machine learning approach of random forest classification, and practiced methods of evaluating the accuracy and validity of project data and results.

Ben Grubbs
GEOG-581

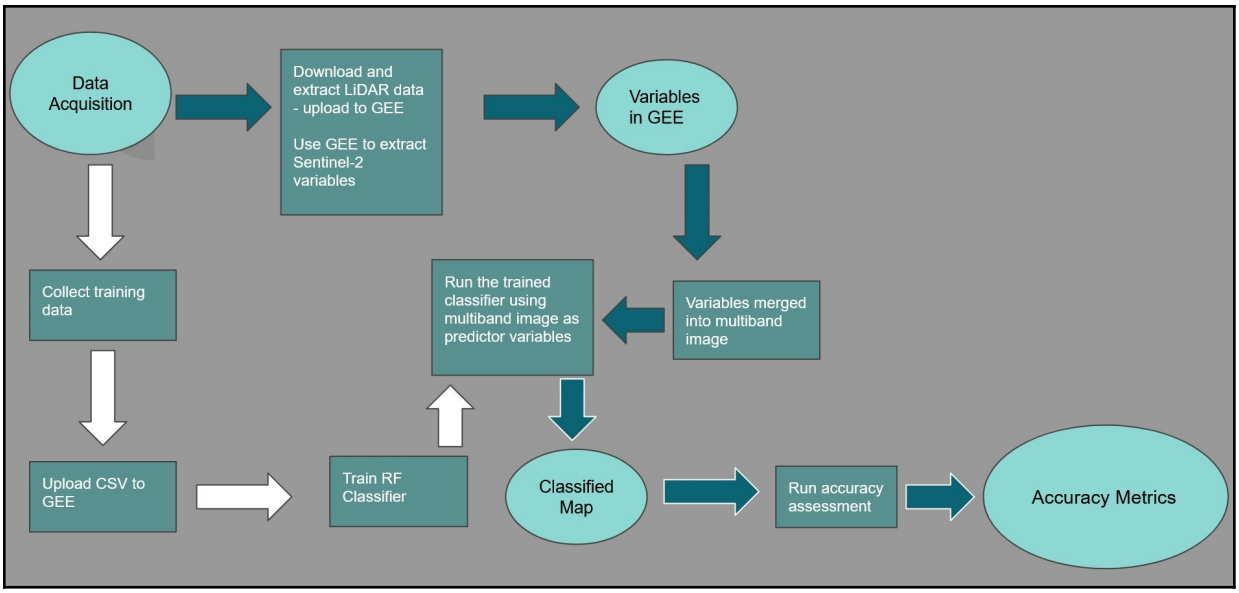
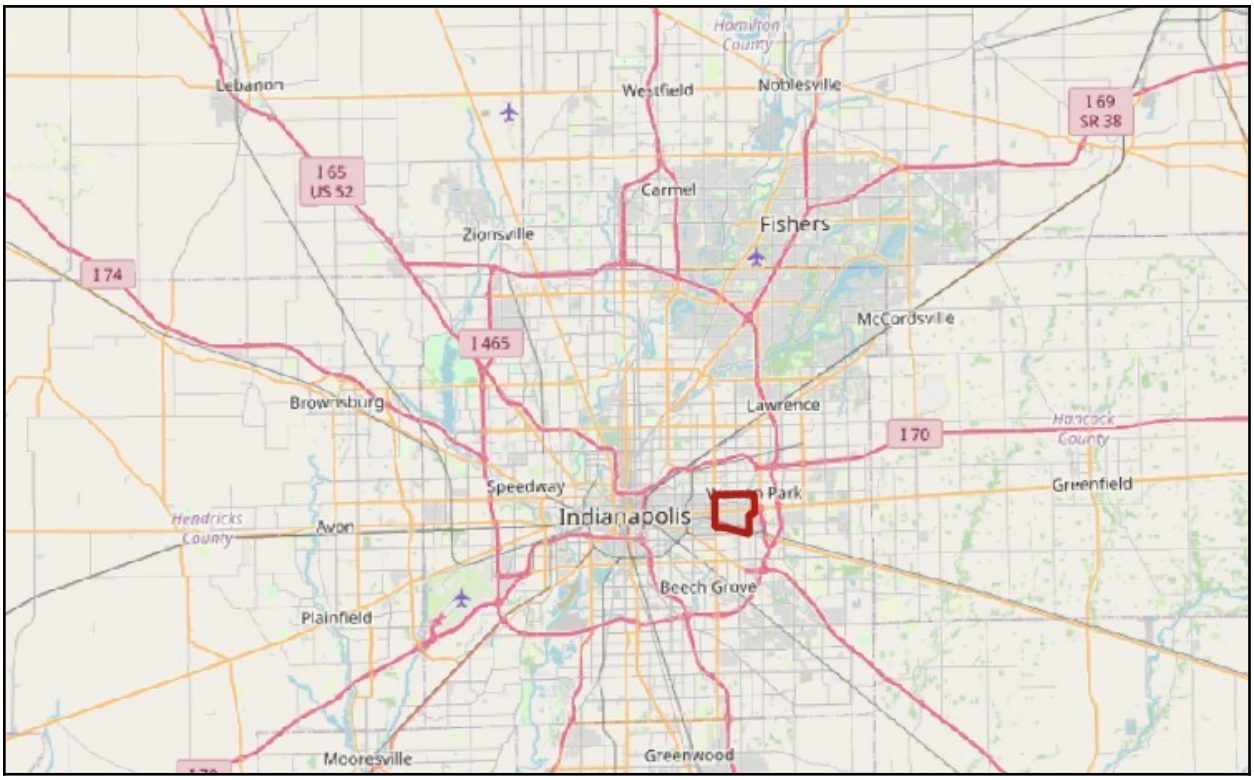
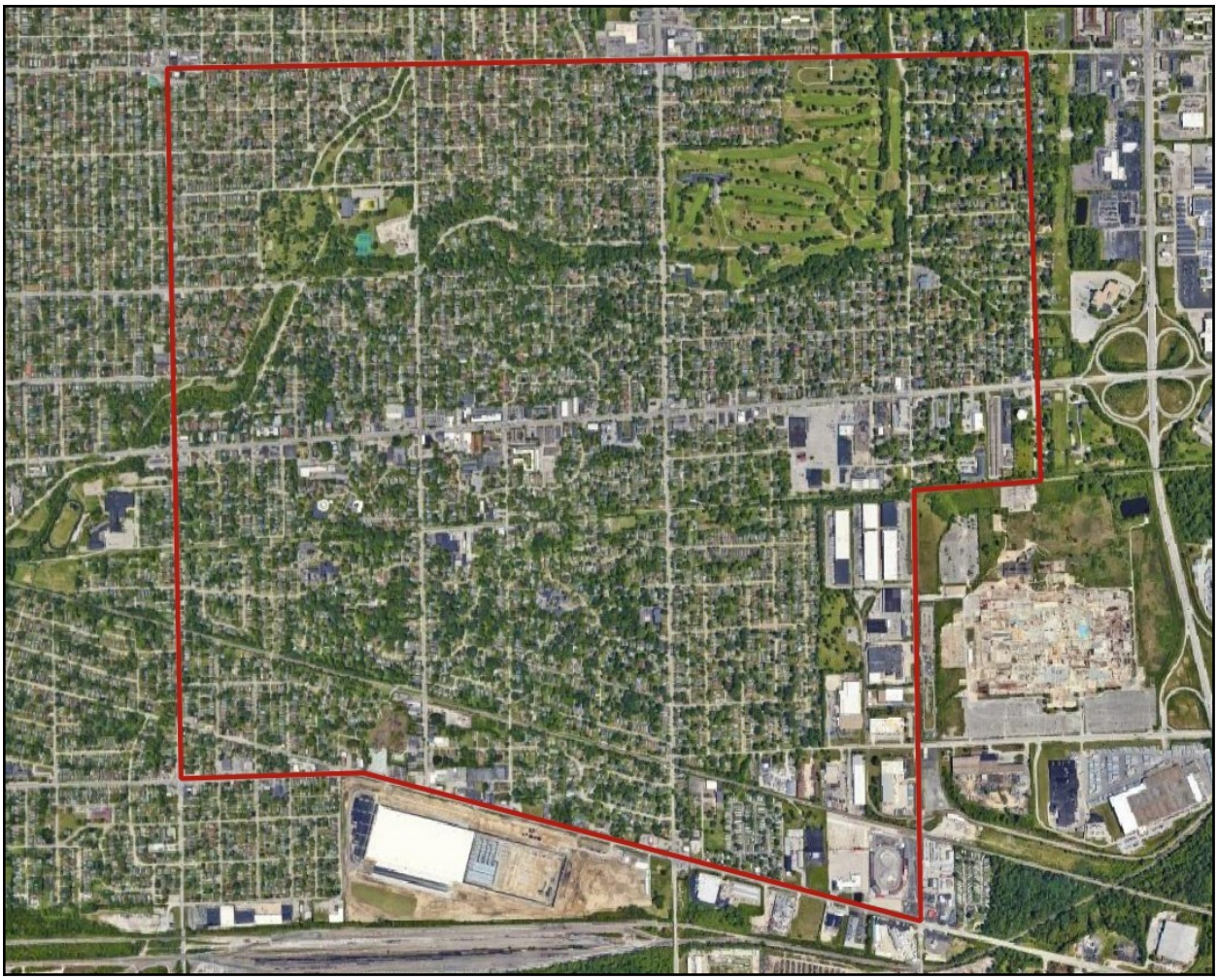



**Figure 1** above shows a flowchart of the project methodology.



Ben Grubbs
GEOG-581



**Figures 2 and 3** above show the project area of the Irvington Historic District in Indianapolis, Indiana.

Ben Grubbs
GEOG-581

```
var digitalElevationModel = digitalElevationModel.select("b1");
var CHM = CHM.select("b1");
var rgbComposite = rgbComposite.select(["b1", "b2", "b3"]);
var aspect = aspect.select("b1");
var slope = slope.select("b1");
var NDVI = NDVI.select("b1");
var NDWI = NDWI.select("b1");
var EVI = EVI.select("b1");
var LAI = LAI.select("b1");
var MNDWI = MNDWI.select("b1");
var NDBI = NDBI.select("b1");

var trainingData = ee.FeatureCollection('projects/ee-bcgrubbs/assets/training_landcover_edited');
Map.centerObject(trainingData, 12);

// Prepare the training data
var labelColumn = "What is the land cover type at this location?"; // Replace with your label column name
var sampleGeometry = "sample_geom"; // Replace with your sample geometry column name

// Convert the label column to a numeric value (0 for "non-tree" and 1 for "trees")
trainingData = trainingData.map(function(feature) {
  var label = ee.String(feature.get(labelColumn));
  var treeClass = ee.Algorithms.If(label.compareTo("trees"), 1, 0);
  return feature.set(labelColumn, treeClass);
});

// Merge the bands into a single image
var mergedImage = CHM
  .addBands(digitalElevationModel)
  .addBands(rgbComposite)
  .addBands(aspect)
  .addBands(slope)
  .addBands(NDVI)
  .addBands(NDWI)
  .addBands(EVI)
  .addBands(LAI)
  .addBands(NDBI)
  .addBands(MNDWI);
```

```
// Train the random forest classifier
var classifier = ee.Classifier.smileRandomForest(100).train({
  features: trainingPoints,
  classProperty: labelColumn,
  inputProperties: inputBands,
});

// Classify the image
var classified = mergedImage.classify(classifier);

// Display the classified image
Map.addLayer(classified, { min: 0, max: 1, palette: ["green", "white"] }, "Classification Result");
```

```
var trainAccuracyRF = v2_trained_RF.confusionMatrix();
print('Resubstitution error matrix RF: ', trainAccuracyRF);
print('Training overall accuracy RF: ', trainAccuracyRF.accuracy());

// For the separate validation, we need to first apply the classifier to our
// points, and then call a method called "errorMatrix" to build the
// confusion matrix from the observations and the map values.

var validationRF = testing_subset.classify(v2_trained_RF);
print(validationRF.first().getInfo());   // this shows us the names of the properties of the feature collection
print(validationRF.aggregate_array('What is the land cover type at this location?'));
print(validationRF.aggregate_array('classification'));

var testAccuracyRF = validationRF.errorMatrix('What is the land cover type at this location?', 'classification');
print('Validation error matrix', testAccuracyRF);
print('Validation overall accuracy: ', testAccuracyRF.accuracy());

// Extract the user and producer accuracy from the confusion matrix
var userAccuracyRF = testAccuracyRF.accuracy();
var producerAccuracyRF = testAccuracyRF.producersAccuracy();

print('User Accuracy RF: ', userAccuracyRF);
print('Producer Accuracy RF: ', producerAccuracyRF);

print('Confusion Matrix:', testAccuracyRF);

// Define a function to calculate precision, recall, and F1 score
var calculateMetrics = function(matrix) {
  var producersAccuracy = matrix.producersAccuracy();
  var consumersAccuracy = matrix.consumersAccuracy();
  var precision = producersAccuracy;
  var recall = consumersAccuracy;
  var f1Score = precision.multiply(recall).multiply(2).divide(precision.add(recall));
  return {
    precision: precision,
    recall: recall,
    f1Score: f1Score
  };
};
```

**Figures 4, 5, and 6** above show the JavaScript code used in GEE to load the variables, combine the variables into a multiband image, prepare the training data, train the classifier, classify the image, display the classified image, then calculate the accuracy assessment metrics.

Ben Grubbs
GEOG-581

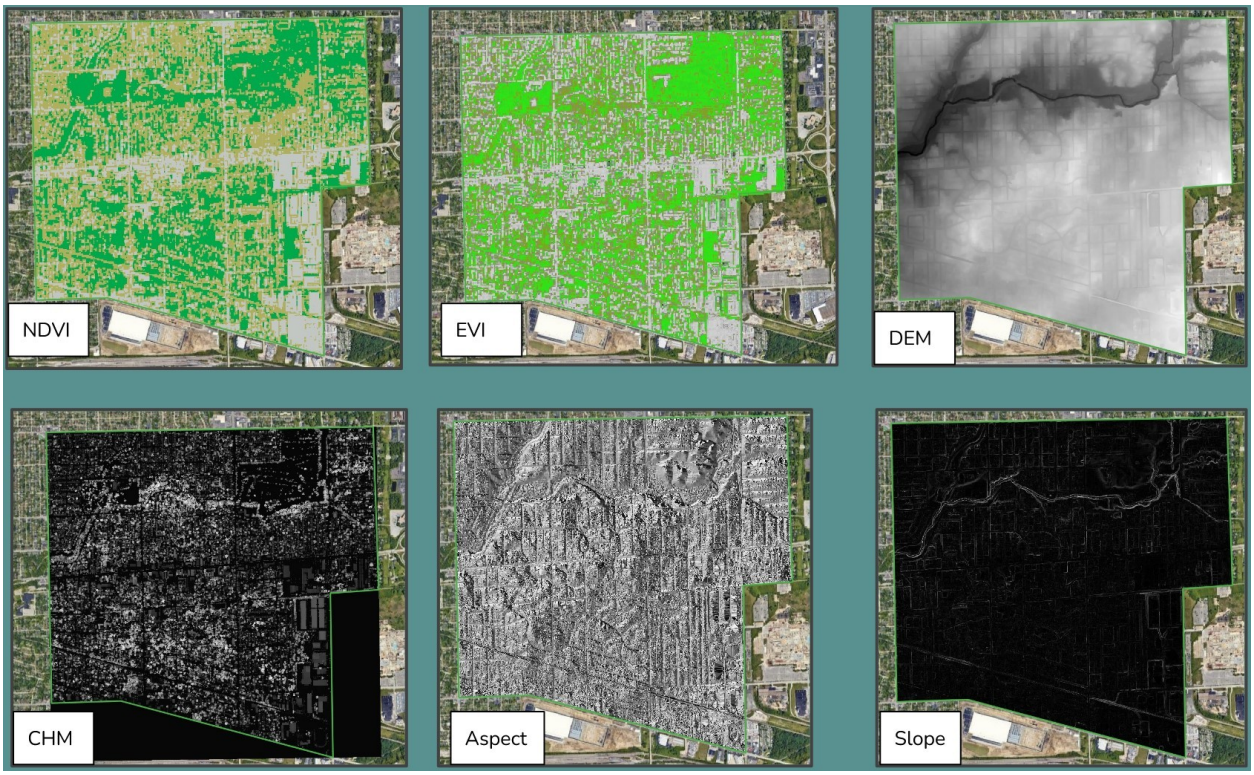



**Figure 7** above displays NDVI, EVI, DEM, CHM, aspect, and slope within the project area.

Ben Grubbs
GEOG-581

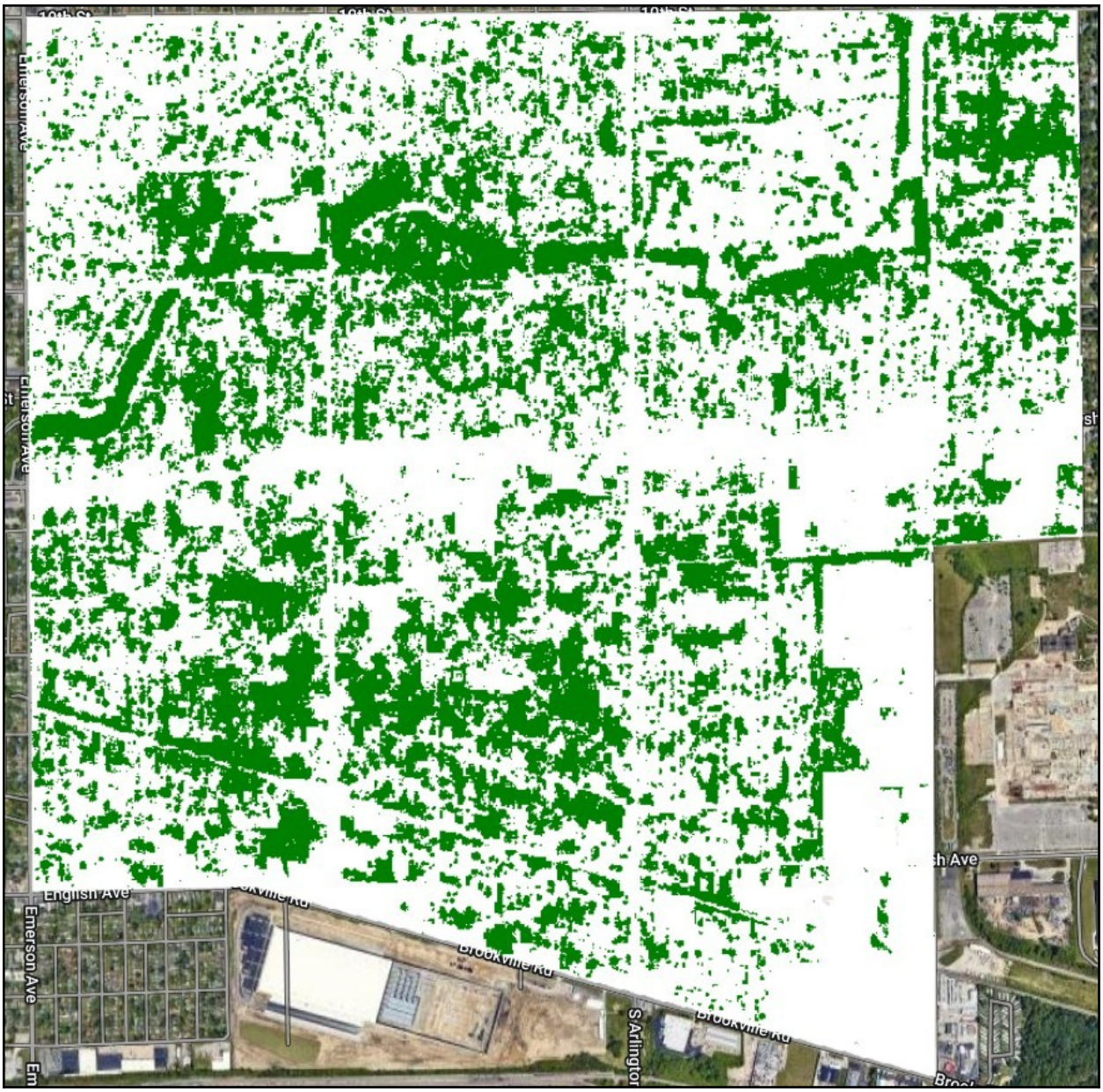



**Figure 8** above shows the final classified image showing tree cover in green and non-tree cover in white.

Ben Grubbs
GEOG-581

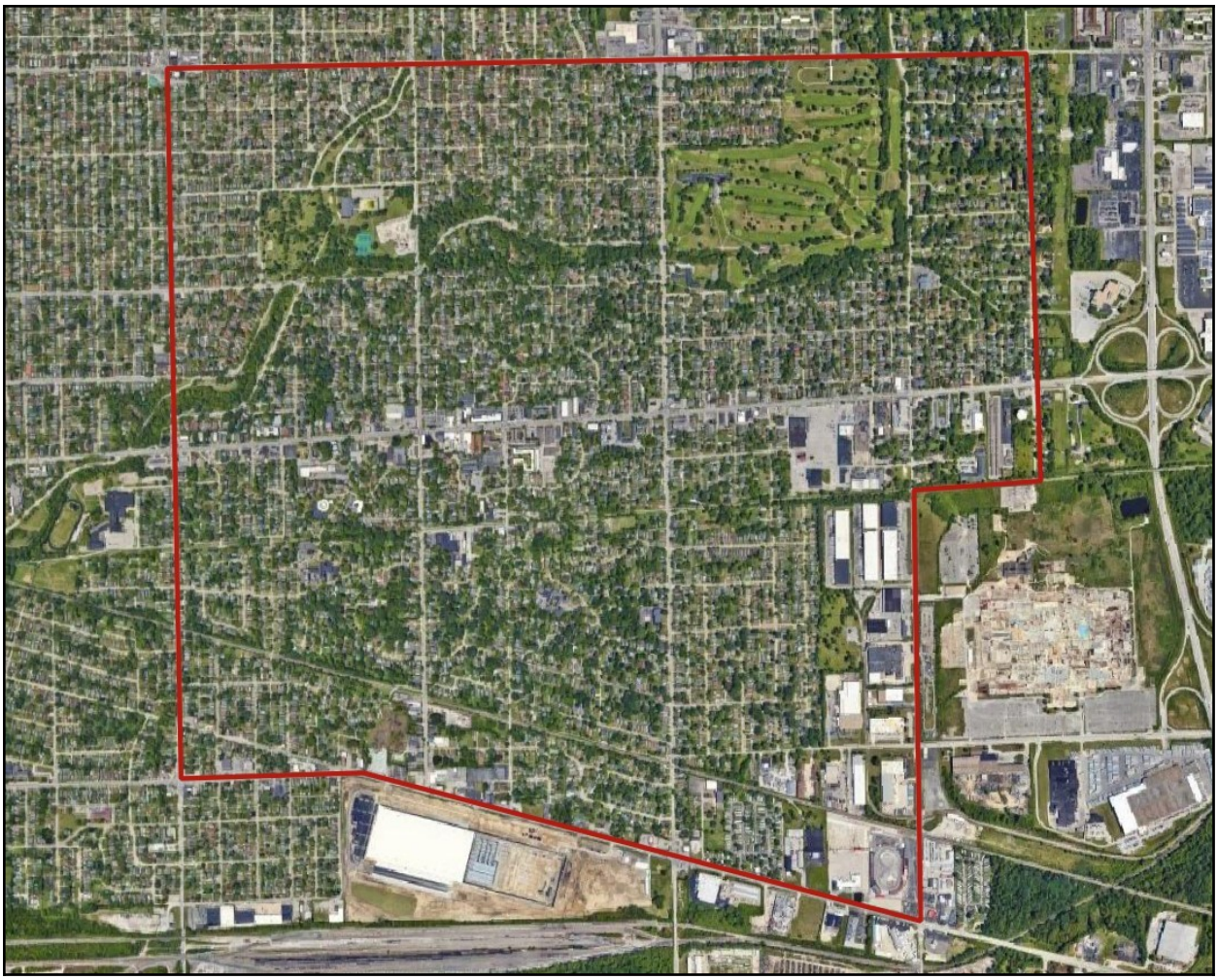
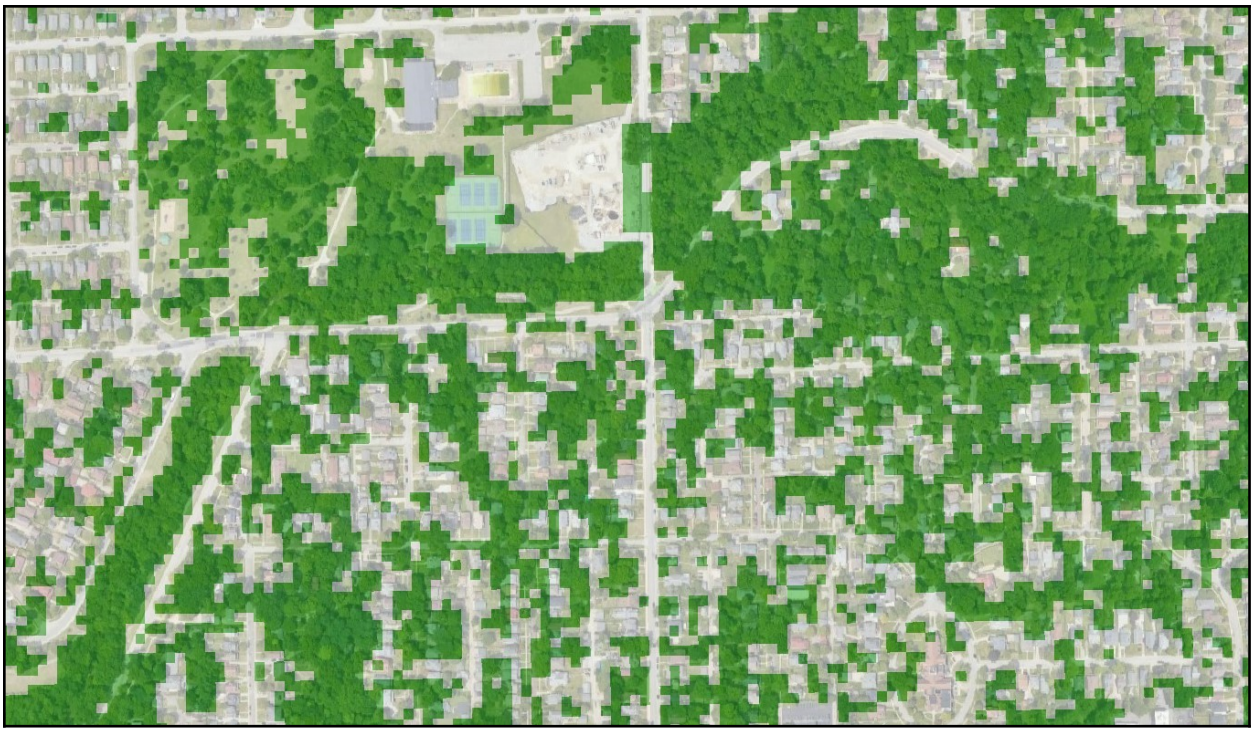


**Figure 9** above shows the Google satellite imagery for the project area for comparison.

Ben Grubbs
GEOG-581



**Figures 10 and 11** above show the classified image zoomed in with a transparency of 50% compared with the same area zoomed in on Google satellite imagery.

Ben Grubbs
GEOG-581

```
Training overall accuracy RF:
0.9689119170984456
```

```
Resubstitution error matrix RF:
[[68,1],[5,119]]
  0: [68,1]
  1: [5,119]
```

```
The size of the training subset is:
193

The size of the testing subset is:
203
```

```
Classifier.train
  type: Classifier.train
  features: FeatureCollection (0 columns)
  classProperty: What is the land cover type at this location?
  classifier: Classifier.smileCart
  inputProperties: List (13 elements)
    0: b1
    1: b1_1
    2: b1_2
    3: b2
    4: b3
    5: b1_3
    6: b1_4
    7: b1_5
    8: b1_6
    9: b1_7
    10: b1_8
    11: b1_9
    12: b1_10
```

Ben Grubbs
GEOG-581

```
Object (14 properties)
  What is the land cover type at this location?: 0
  b1: 18.354244232177734
  b1_1: 806.500732421875
  b1_10: -0.49766412377357483
  b1_2: 41
  b1_3: 137.68368530273438
  b1_4: 1.7901781797409058
  b1_5: 0.2689545154571533
  b1_6: -0.6254863739013672
  b1_7: 2.3889305817760027
  b1_8: 2.702076220578089
  b1_9: -0.21562211215496063
  b2: 53
  b3: 37
```

```
Producer Accuracy RF:
[[0.7246376811594203],[0.9701492537313433]]
  0: [0.7246376811594203]
  1: [0.9701492537313433]
```

```
Validation overall accuracy:
0.8866995073891626
```

```
Validation error matrix
[[50,19],[4,130]]
  0: [50,19]
  1: [4,130]
```

**Figures 12-19** above show the accuracy metrics results for this project.

Ben Grubbs
GEOG-581

**References**

Belgiu, M., & Dragut, L. (2016). Random forest in remote sensing: A review of applications and future directions. *ISPRS Journal of Photogrammetry and Remote Sensing*. https://www.sciencedirect.com/science/article/abs/pii/S0924271616000265?via%3Dihub

Dietterich, T. (2000). nsemble Metho ds in Machine Learning. *Oregon State University Corvallis Oregon US*. https://link.springer.com/chapter/10.1007/3-540-45014-9_1

Ebadat, P., Amati, M., & Fraser, C. (2016). MAPPING URBAN TREE CANOPY COVER USING FUSED AIRBORNE LIDAR AND SATELLITE IMAGERY DATA. *ISPRS Annals of the Photogrammetry*. https://www.researchgate.net/profile/Ebadat-Ghanbari-Parmehr/publication/307530163_MAPPING_URBAN_TREE_CANOPY_COVER_USING_FUSED_AIRBORNE_LIDAR_AND_SATELLITE_IMAGERY_DATA/links/57d9d7b908ae0c0081efbeb1/MAPPING-URBAN-TREE-CANOPY-COVER-USING-FUSED-AIRBORNE-LID

Li, X., Chen, W., Sanesi, G., & Lafortezza, R. (2019). Remote Sensing in Urban Forestry: Recent Applications and Future Directions. *remote sensing*. www.mdpi.com/journal/remotesensing

Speiser, J., Miller, M., Tooze, J., & Ip, E. (2019). A comparison of random forest variable selection methods for classification prediction modeling. *Expert Systems With Applications*. https://www.sciencedirect.com/science/article/abs/pii/S0957417419303574?via%3Dihub

Wai, Y., Ahmed, S., & El-Ashmawy, N. (2015). Urban land cover classification using airborne LiDAR data: A review. *Remote Sensing of Environment*. https://www.sciencedirect.com/science/article/pii/S0034425714004374?via%3Dihub

Zhang, C., Zhou, Y., & Qiu, F. (2015). Individual Tree Segmentation from LiDAR Point Clouds for Urban Forest Inventory. *remote sensing*. www.mdpi.com/journal/remotesensing